CS3500 Operating Systems Lab
Assignment on Xv6 System Calls

~~Due: 11:59 PM, 20th August, 2026~~
Due: 11:59 PM, 23rd August, 2026

Xv6 Book: https://pdos.csail.mit.edu/6.1810/2026/xv6/book-riscv-rev5.pdf

1 Introduction

In the previous lab you used system calls to write a few utilities. In this lab you will add a new system call to xv6, which will help you understand how system calls work and will expose you to some of the internals of the xv6 kernel. You will add more system calls in later labs.

1.1 Background reading

Before you start coding, read Chapter 2 of the xv6 book, and Sections 4.3 and 4.4 of Chapter 4, along with the related source files:

- The user-space "stubs" that route system calls into the kernel are in `user/usys.S`, which is generated by `user/usys.pl` when you run `make`. Declarations are in `user/user.h`.
- The kernel-space code that routes a system call to the kernel function that implements it is in `kernel/syscall.c` and `kernel/syscall.h`.
- Process-related code is in `kernel/proc.h` and `kernel/proc.c`.

1.2 The system-call path in xv6

A system call is the controlled interface through which a user program requests a service from the operating-system kernel. User programs cannot directly execute privileged kernel operations or access kernel data structures; instead, a system-call request crosses the user/kernel protection boundary and is dispatched by the kernel. On xv6/RISC-V, the path a system call takes is:

user program → user stub → `ecall` → trap handling → `syscall()` → kernel implementation

On RISC-V, the system-call number is placed in register `a7`; arguments are passed in registers and retrieved by kernel helper functions. The `ecall` instruction transfers control to the kernel, after which xv6 uses the saved trap frame to identify and execute the requested call.

2 Xv6 System Calls

Table 1 lists the standard system calls present in the xv6 RISC-V source tree, together with a short description of what each one does.

System Call (Prototype)	Purpose
`int fork()`	Create a process; return the child's PID.

Continued on next page

Table 1 – continued from previous page

System Call (Prototype)	Purpose
`int exit(int status)`	Terminate the current process; `status` is reported to `wait()`. No return.
`int wait(int *status)`	Wait for a child to exit; exit status stored in `*status`; returns child PID.
`int kill(int pid)`	Terminate process `pid`; returns 0, or −1 on error.
`int getpid()`	Return the current process's PID.
`int sleep(int n)`	Pause for `n` clock ticks.
`int exec(char *file, char *argv[])`	Load a file and execute it with arguments; returns only on error.
`char *sbrk(int n)`	Grow the process's memory by `n` bytes; returns the start of the new memory.
`int open(char *file, int flags)`	Open a file; `flags` indicate read/write access; returns a file descriptor.
`int write(int fd, char *buf, int n)`	Write `n` bytes from `buf` to file descriptor `fd`; returns `n`.
`int read(int fd, char *buf, int n)`	Read `n` bytes into `buf`; returns the number read, or 0 at end of file.
`int close(int fd)`	Release the open file descriptor `fd`.
`int dup(int fd)`	Return a new file descriptor referring to the same file as `fd`.
`int pipe(int p[])`	Create a pipe; put the read/write file descriptors in `p[0]` and `p[1]`.
`int chdir(char *dir)`	Change the current directory.
`int mkdir(char *dir)`	Create a new directory.
`int mknod(char *file, int, int)`	Create a device file.
`int fstat(int fd, struct stat *st)`	Place information about an open file into `*st`.
`int link(char *file1, char *file2)`	Create another name (`file2`) for the file `file1`.
`int unlink(char *file)`	Remove a file.

Table 1: xv6 system calls and their purposes.

3 How to Create a System Call in Xv6

Adding a system call requires connecting the user-space interface to a kernel implementation. In general, the following steps are needed:

1. **Define a system-call number:** add `SYS_name` to `kernel/syscall.h`.
2. **Declare the user function:** add its prototype to `user/user.h`.
3. **Add the user stub:** add the call to `user/usys.pl`. Running `make` generates `user/usys.S`; the stub loads the system-call number into register `a7` and executes `ecall`.
4. **Implement the kernel function:** normally add `sys_name()` in an appropriate kernel source file such as `kernel/sysproc.c`. Retrieve arguments using xv6's argument helper functions (e.g. `argint`, `argaddr`, `argstr`), which are defined in `kernel/syscall.c`.

5. **Register the implementation:** add `sys_name` to the `syscalls[]` dispatch table in `kernel/syscall.c`.
6. **Add user programs/tests if required:** update the `Makefile` (for example, `$U/_program`) and test the call from the xv6 shell.

For this lab, the new system call is `interpose(mask, path)`. Its `mask` identifies system calls that should be rejected for the calling process. This restriction must also be inherited by children created with `fork()`.

4 Setting Up for "syscall" lab

In the official git repository `git://g.csail.mit.edu/xv6-labs-2025`, the labs are organized into different branches, and that repository's compiler warnings are treated strictly – the build expects all warnings to be resolved.

The patched version used for this course can instead be cloned from:

https://github.com/anupamsworld/IITM_CS3500_Xv6_Lab_2026.git

Note: Unlike the official repository structure, in the modified repository provided for this course you do *not* have to switch branches for different labs (Utilities, System Calls, etc.). Instead, a separate directory exists at the root of the repository for each lab. All that is required is to point Docker at the right directory when starting the container. Example:

```
$ git clone https://github.com/anupamsworld/IITM_CS3500_Xv6_Lab_2026.git
$ docker run -it -v <path-to-IITM_CS3500_Xv6_Lab_2026-repo>/syscall:/home/xv6-labs \
     nandhagk/xv6-tools:latest
```

This starts the Docker container with the syscall lab mounted and ready to use.

5 Assignment Tasks

5.1 Using GDB *(easy)*

GDB (GNU Debugger) can be used to examine the execution of xv6, inspect variables and registers, set breakpoints, and obtain stack backtraces. In this exercise, GDB is used to understand the path followed by a system call from the user program to the xv6 kernel.

5.1.1 Starting xv6 with GDB

Start xv6 in GDB debugging mode in one docker container terminal:

```
bash-5.3# make qemu-gdb
```

Then, in another terminal, start a RISC-V-compatible version of GDB:

Note: As we are using docker container, open two different terminals in the same docker container, using container id.

To know the already running container's id:

```
bash-5.3# cat /etc/hostname
```

Then,

```
$ docker exec -it <container id> bash
bash-5.3# riscv64-linux-gnu-gdb
```

Depending on the installation, `riscv64-linux-gnu-gdb` or **`riscv64-unknown-elf-gdb`** may also be used. Set a breakpoint and use `c` (continue) to allow xv6 to execute until the breakpoint is reached.

5.1.2 Tracing a System Call

Set a breakpoint at the xv6 `syscall()` function:

```
(gdb) b syscall
Breakpoint 1 at 0x80002142: file kernel/syscall.c, line 243.
(gdb) c
Continuing.
[Switching to Thread 1.2]

Thread 2 hit Breakpoint 1, syscall () at kernel/syscall.c:243
243 {
```

When the breakpoint is reached, inspect the source code using:

```
(gdb) layout src
```

Use the following command to display the stack backtrace:

```
(gdb) backtrace
```

The `layout src` command splits the window in two, showing where GDB is in the source code. `backtrace` prints a stack backtrace.

Answer the following questions in `answers-syscall.txt`:

1. Looking at the backtrace output, which function called `syscall()`?
2. Type `n` a few times to step past the statement `struct proc *p = myproc();`. Once past this statement, type `p /x *p`, which prints the current process's `proc` struct (see `kernel/proc.h`) in hex. What is the value of `p->trapframe->a7` and what does that value represent? *(Hint: look at `user/init.c`, the first user program xv6 starts, and its compiled assembly `user/init.asm`.)*
3. The processor is running in supervisor mode, and privileged registers such as `sstatus` can be printed:

   ```
   (gdb) p /x $sstatus
   ```

   What was the previous mode that the CPU was in?
4. The xv6 kernel code contains consistency checks whose failure causes the kernel to panic; you may find that your kernel modifications cause panics. For example, replace the statement `num = p->trapframe->a7;` with `num = *(int *)0;` at the beginning of `syscall`, run `make qemu`, and you will see something similar to:

   ```
   xv6 kernel is booting

   hart 2 starting
   hart 1 starting
   scause=0xd sepc=0x80001bfe stval=0x0
   panic: kerneltrap
   ```

Quit out of QEMU.

5. To track down the source of a kernel page-fault panic, search for the `sepc` value printed for the panic above in the file `kernel/kernel.asm`, which contains the assembly for the compiled kernel. Write down the assembly instruction the kernel is panicking at. Which register corresponds to the variable `num`?
6. To inspect the state of the processor and the kernel at the faulting instruction, fire up GDB and set a breakpoint at the faulting `epc`, like this:

```
(gdb) b *0x80001bfe
Breakpoint 1 at 0x80001bfe: file kernel/syscall.c, line 138.
(gdb) layout asm
(gdb) c
Continuing.
[Switching to Thread 1.3]

Thread 3 hit Breakpoint 1, syscall () at kernel/syscall.c:138
```

   Confirm that the faulting assembly instruction is the same as the one you found above.
7. Why does the kernel crash? *Hint: look at Figure 3-3 in the xv6 text – is address 0 mapped in the kernel address space? Is that confirmed by the value in* `scause` *above?* Relate the exception to the attempted access to address 0 and the RISC-V page-fault mechanism.
8. Note that `scause` was printed by the kernel panic above, but often additional information is needed to track down the process that caused the panic. For example, to find out which user process was running when the kernel panicked, print the process's name:

```
(gdb) p p->name
```

   What is the name of the process that was running when the kernel panicked? What is its process ID (PID)?

5.1.3 Useful GDB and Debugging Commands

The following commands are useful when debugging xv6. If the kernel panics unexpectedly, a useful approach is to set a breakpoint at `panic`:

```
(gdb) b panic
(gdb) c
```

When the breakpoint is reached, use:

```
(gdb) bt
```

to inspect the sequence of function calls that led to the panic.

The file `kernel/kernel.asm`, generated during kernel compilation, can also be used to inspect the assembly instructions corresponding to kernel addresses. Alternatively, a program-counter value can be translated to a source location using:

```
$ addr2line -e kernel/kernel <pc-value>
```

You may want to revisit "Using the GNU Debugger" as needed. The course guidance page also has debugging tips. https://pdos.csail.mit.edu/6.1810/2026/labs/guidance.html

5.2 Sandbox a Command

In this assignment problem you will "sandbox" a process to restrict the system calls it can make. For example, one might sandbox a process to disallow opening files. You will create a new `interpose` system call that specifies which system calls the kernel should reject from the calling process. `interpose` takes two arguments: an integer `mask` and a `path`. The mask's bits specify which system calls to reject. The second argument will be used in the next assignment problem; in this assignment problem it is always `"-"`.

For example, in order for a process to prevent itself from using the `open` system call, it should call

```
interpose(1 << SYS_open, "-")
```

where `SYS_open` is a syscall number from `kernel/syscall.h`. Your implementation should cause the mask to be inherited by children of `fork`, so that children inherit the parent's restrictions.

You are provided with a `user/sandbox.c` user program that forks, calls `interpose()` in the child, and then `exec`s a program in the child. When you are done implementing `interpose()`, you should see output like this:

```
$ sandbox 32768 - cat README
cat: cannot open README
$
```

In this example, `cat README` is the command being sandboxed. `32768` is the mask of system calls to reject; in this example it is `1<<SYS_open`. If your solution is correct, you should see "cat: cannot open README".

Your solution gets full points if it passes the test in `grade-lab-syscall sandbox_mask`:

```
$ ./grade-lab-syscall sandbox_mask
== Test sandbox_mask == sandbox_mask: OK (1.5s)
```

5.2.1 Hints

- Add `$U/_sandbox` to `UPROGS` in the `Makefile`.
- Run `make qemu` and you will see that the compiler cannot compile `user/sandbox.c`, because the user-space stubs for the `interpose` system call do not exist yet: add a prototype for `interpose` to `user/user.h`, a stub to `user/usys.pl`, and a syscall number to `kernel/syscall.h`. The Makefile invokes the Perl script `user/usys.pl`, which produces `user/usys.S`, the actual system-call stubs, which use the RISC-V `ecall` instruction to transition to the kernel. Once you fix the compilation issues, run `sandbox 32768 - cat README` in the xv6 shell; it will fail because you have not implemented the system call in the kernel yet.
- Add a `sys_interpose()` function in `kernel/sysproc.c` that implements the new system call by recording the mask argument in a new field in the `proc` structure (see `kernel/proc.h`). The functions to retrieve system-call arguments from user space are in `kernel/syscall.c`, and you can see examples of their use in `kernel/sysproc.c`. Add your new `sys_interpose` to the `syscalls` array in `kernel/syscall.c`.
- Modify `kfork()` (see `kernel/proc.c`) to copy the mask from the parent to the child process.

- Modify the `syscall()` function in `kernel/syscall.c` to check whether the system call must be rejected.

5.3 Sandbox with Allowed Pathnames

In this assignment problem you will extend the sandbox to allow masked `open` and `exec` system calls based on the pathname they use. The second argument of `sys_interpose` is the pathname allowed. If `open` or `exec` is masked but the pathname matches the allowed pathname, then these system calls should be allowed.

When you are done, you should see output like this:

```
$ sandbox 32768 README grep xv6 README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6). xv6 loosely follows the structure and style of v6,
xv6 is inspired by John Lions's Commentary on UNIX 6th Edition (Peer
(kaashoek,rtm@mit.edu). The main purpose of xv6 is as a teaching
$ sandbox 32768 README grep xv6 x
grep: cannot open x
```

In this example, the sandbox allows `open` calls that access `README`, but not any other file (e.g., `x`).

Your solution is correct if it passes the sandbox tests in `make grade`:

```
$ make grade
...
== Test sandbox_mask ==
$ make qemu-gdb
sandbox_mask: OK (9.5s)
== Test sandbox_fork ==
$ make qemu-gdb
sandbox_fork: OK (0.3s)
== Test sandbox_path ==
$ make qemu-gdb
sandbox_path: OK (1.1s)
== Test sandbox_most ==
$ make qemu-gdb
sandbox_most: OK (0.8s)
== Test sandbox_minus ==
$ make qemu-gdb
sandbox_minus: OK (1.1s)
...
```

5.3.1 Hints

- Modify `sys_interpose()` to remember the allowed pathname. `argstr` will be handy to retrieve the pathname. You can declare a buffer of size `MAXPATH` in the `proc` struct.
- If `open` or `exec` are masked, check whether the pathname matches the allowed pathname. If so, allow the execution of those system calls.

5.4 Attack xv6

The xv6 kernel isolates user programs from each other and isolates the kernel from user programs. As you saw in the above assignments, an application cannot directly call a function in the kernel or in another user program; instead, interactions occur only through system calls.

However, if there is a bug in the kernel's implementation of a system call, an attacker may be able to exploit that bug to break the isolation boundaries. To get a sense of how bugs can be exploited, a bug has been introduced into xv6, and your goal is to exploit that bug to steal a secret from another process.

The bug is that the call to `memset(mem, 0, sz)` in `uvmalloc()` in `kernel/vm.c`, which clears a newly-allocated page, is omitted when compiling this lab. Similarly, when compiling `kernel/kalloc.c` for this lab, the two lines that use `memset` to put garbage into free pages are omitted. The net effect of omitting these three lines (all marked by `ifndef LAB_SYSCALL`) is that newly allocated memory retains the contents from its previous use. Thus an application that calls `sbrk()` to allocate memory may receive pages that have data in them from previous uses. Despite the three deleted lines, xv6 mostly works correctly; it even passes most of `usertests`.

`user/secret.c` writes a secret string into its memory and then exits (which frees its memory). Your goal is to add a few lines of code to `user/attack.c` to find the secret that a previous execution of `secret.c` wrote to memory, and to print the secret on a line by itself.

Your `attack.c` must work with unmodified xv6 and unmodified `secret.c`. You can change anything to help you experiment and debug, but you must revert those changes before final testing and submitting.

The `secret` program takes the secret as an argument. You can test your attack program by running `secret` with some argument, then running `attack`, and seeing whether `attack` prints exactly the argument passed to `secret`. Here is a successful run:

```
$ secret xyzzy
$ attack
xyzzy
$
```

Depending on exactly how you implement your attack, you may need to run `attack` a second time in order for it to find the secret. The grader runs `attack` twice, just in case, and is satisfied if either run produces the secret.

From outside xv6, you can use `./grade-lab-syscall attack`, or `make grade`, to see if your attack passes the tests. The secret strings that the tests generate are guaranteed to contain only digits and upper- and lower-case letters.

As with this example, bugs that do not directly affect correctness can sometimes be exploited to break security. Careful programming and extensive testing can reduce the number of bugs, but cannot guarantee their absence. xv6 has had bugs in the past, and probably has yet more undiscovered errors. Real kernels, which have much more code than xv6, have a long history of such bugs – see, for example, the public Linux kernel vulnerability listings and guidance on how to report kernel security bugs.

6 Submission and Optional Challenge

6.1 Time spent

Create a new file, `time.txt`, and put in it a single integer – the number of hours you spent on the lab. Run `git add` and `git commit` on the file.

6.2 Answers

If this lab had questions, write up your answers in `answers-*.txt` (i.e. `answers-syscall.txt`). Run `git add` and `git commit` on these files.

6.3 Submit

Assignment submissions are handled via Moodle (for this course). When you are ready to submit, run:

```
$ make zipball
```

which generates `lab.zip`. If you have either uncommitted changes or untracked files, you will see output similar to the following:

```
 M hello.c
?? bar.c
?? foo.pyc
Untracked files will not be handed in. Continue? [y/N]
```

Inspect the above lines and make sure all files that your lab solution needs are tracked, i.e., not listed on a line that begins with `??`. You can cause git to track a new file that you create using `git add {filename}`.

- Please run `make grade` to ensure that your code passes all of the tests. The autograder will use the same grading program to assign your submission a grade.
- Commit any modified source code before running `make zipball`.
- Rename the resulting `lab.zip` to `<ROLL NUMBER>_OS_LAB_2_SystemCall.zip` and upload it to Moodle.

6.4 Optional Challenge Exercise

Find a bug in xv6 that allows an adversary to break process isolation or crash the kernel, and report it. Side-channel attacks such as Meltdown are out of scope, although they may be covered in lecture.

Sources: MIT 6.1810 System Calls Lab (Fall 2026), https://pdos.csail.mit.edu/6.1810/2026/labs/syscall.html; MIT xv6-RISC-V source tree, https://github.com/mit-pdos/xv6-riscv.